

JobSwipe — Présentation du travail backend

1. Objectif du projet

JobSwipe est une plateforme de matching professionnel inspirée du principe d'intérêt réciproque de Tinder :

- Le candidat consulte les offres sous forme de cartes.
- Il passe une offre ou manifeste son intérêt.
- Le recruteur ne consulte que les candidats intéressés par ses propres offres.
- Il examine le profil, la photo et le CV du candidat.
- Il accepte ou refuse la candidature.
- L'acceptation crée le match réciproque et ouvre une conversation.

Le backend fournit l'API REST, la persistance MongoDB, la messagerie temps réel, les notifications et les tests automatisés.

2. Analyse du prototype HTML

Le fichier **jobswipe.html** a servi de source de besoins. Il contient deux parcours :

- le parcours candidat : inscription, profil, préférences, découverte, intérêt, matchs et conversation ;
- le parcours recruteur : entreprise, publication d'offres, file de candidats, décision et conversations.

Cette analyse a permis d'identifier les champs des profils, les informations des offres, le CV, les photos, les logos, les matchs, les messages et les notifications.

La notion importante est l'ancrage fonctionnel : chaque écran du prototype doit correspondre à une donnée persistée ou à une route backend identifiable.

3. Architecture backend

Le projet utilise une architecture Express organisée par responsabilités :

- **models/** : schémas MongoDB avec Mongoose ;
- **controllers/** : logique métier des requêtes ;
- **routes/** : définition des endpoints ;
- **middlewares/** : authentification, rôles et uploads ;
- **services/** : services réutilisables, notamment les notifications ;

- **sockets/** : événements Socket.IO ;
- **utils/** : JWT, stockage média, score et complétion ;
- **tests/** : tests unitaires et intégration.

Cette séparation applique la notion de responsabilité unique : chaque module décide d'un seul type de comportement.

4. Modèle de données MongoDB

Les modèles principaux sont :

- **User** : compte, rôle et profil candidat ou recruteur embarqué ;
- **JobOffer** : offre publiée par un recruteur ;
- **Match** : intérêt candidat lié à une offre, avec statut et score ;
- **Conversation** : conversation ouverte après acceptation ;
- **Message** : message envoyé dans une conversation ;
- **Notification** : événement destiné à un utilisateur.

Le profil est embarqué dans **User**, car le MVP ne nécessite pas de collection séparée pour les profils. Le choix MongoDB favorise ici les documents imbriqués pour les informations consultées avec le compte.

Les offres, matchs, conversations et messages utilisent des références MongoDB, car ce sont des entités indépendantes avec leur propre cycle de vie.

5. Authentification et autorisation

Notions

L'authentification répond à « qui est l'utilisateur ? ». L'autorisation répond à « que peut-il faire ? ».

Le système utilise :

- **bcryptjs** pour hasher les mots de passe ;
- JWT pour représenter la session ;
- le header **Authorization: Bearer <token>** ;
- **protect** pour vérifier le token ;
- **requireRole** pour limiter les routes aux candidats ou recruteurs.

Fonctionnement

Lors de l'inscription ou de la connexion, le backend renvoie un JWT. Pour chaque route protégée, le middleware vérifie le token, recharge l'utilisateur MongoDB et place le résultat dans **request.user**.

Le mot de passe hashé n'est jamais renvoyé dans les réponses.

6. Gestion des profils et préférences

Profil candidat

Le candidat peut enregistrer :

- identité professionnelle ;
- titre et expérience ;
- localisation ;
- disponibilité ;
- mode de travail ;
- types de contrats recherchés ;
- salaire minimum et maximum ;
- compétences ;
- langues ;
- formations multiples ;
- biographie ;
- CV ;
- photo.

Profil recruteur

Le recruteur peut enregistrer :

- entreprise ;
- secteur ;
- taille ;
- ville ;
- identité et fonction ;
- délai de réponse ;
- logo de l'entreprise.

Complétion

Le backend calcule un pourcentage **profileCompletion** entre 0 et 100. Cette valeur est renvoyée lors de la consultation et de la modification du profil.

La notion utilisée est une validation progressive : le profil peut être créé partiellement, puis complété étape par étape.

7. CV, photo et logo

Les fichiers sont reçus avec **multipart/form-data** et traités par Multer.

CV

- PDF uniquement ;
- 5 Mo maximum ;
- stockage local protégé ;
- remplacement du fichier précédent ;
- téléchargement candidat authentifié ;

- téléchargement recruteur uniquement via un match dont il est propriétaire.

Photo et logo

- photo candidat : PNG ou JPEG, 2 Mo maximum ;
- logo recruteur : PNG, JPEG ou SVG, 2 Mo maximum ;
- fichiers non exposés par une route publique ;
- accès contrôlé par le rôle et la relation métier.

La notion importante est la séparation entre l'URL métier enregistrée dans MongoDB et le chemin physique du fichier sur le serveur.

8. Offres d'emploi

Le recruteur peut :

- créer une offre ;
- consulter ses offres ;
- modifier ses offres ;
- fermer ou republier une offre ;
- supprimer une offre.

Le candidat peut :

- lister les offres publiées ;
- consulter le détail d'une offre ;
- filtrer par ville, contrat, mode de travail et compétence ;
- utiliser la pagination ;
- voir le logo de l'entreprise liée à une offre publiée.

Le contrôle de propriété empêche un recruteur de modifier l'offre d'un autre recruteur.

9. Matching et intérêt réciproque

Le matching suit le cycle :

pending → accepted
pending → rejected

pending signifie que le candidat a manifesté son intérêt. Ce n'est pas encore un match réciproque.

Le recruteur peut consulter le profil, la photo et le CV avant de prendre sa décision. Lorsque le statut devient **accepted**, le backend crée la conversation et notifie le candidat.

Une contrainte unique empêche un candidat de manifester deux fois son intérêt pour la même offre.

10. Score de compatibilité

Le score est calculé côté backend à partir du profil candidat et de l'offre :

- compétences : 40 % ;
- localisation et mode de travail : 20 % ;

- type de contrat : 15 % ;
- langues : 10 % ;
- expérience : 10 % ;
- diplôme : 5 %.

Le score apparaît dans les offres consultées par le candidat et est sauvegardé dans le **Match** au moment de l'intérêt.

Cette règle garantit que le frontend ne peut pas falsifier le score.

11. Conversations et Socket.IO

Une conversation est créée uniquement après l'acceptation du recruteur.

Les routes HTTP permettent de :

- lister les conversations ;
- lire les messages ;
- envoyer un message ;
- compter les messages non lus ;
- marquer une conversation comme lue.

Socket.IO apporte le temps réel :

- authentification JWT au handshake ;
- room privée par utilisateur ;
- room dédiée par conversation ;
- **conversation:join** ;
- **message:send** ;
- **message:new** ;
- **conversation:read** ;
- **message:read**.

Les messages sont sauvegardés dans MongoDB avant leur diffusion. Socket.IO accélère l'échange, mais ne remplace pas la persistance.

12. Notifications

Le modèle **Notification** conserve :

- le destinataire ;
- l'auteur de l'action ;
- le type ;
- le titre ;
- le message ;
- les données métier ;
- la date de lecture.

Les notifications sont créées pour :

- un nouveau candidat intéressé ;
- une candidature acceptée ;
- une candidature refusée ;
- un nouveau message.

Elles sont disponibles par API et diffusées en temps réel avec **notification:new**.

13. Documentation et CI

Swagger/OpenAPI est accessible avec :

`http://localhost:3000/api-docs`

Le JSON est disponible sur :

`http://localhost:3000/api-docs.json`

La pipeline GitHub Actions :

- installe Node.js 20 ;
- démarre MongoDB 7 ;
- exécute **npm ci** ;
- lance **npm test**.

Elle se déclenche sur les push et pull requests vers **develop**.

14. Tests automatisés

La suite vérifie actuellement :

- JWT et authentification ;
- inscription et connexion ;
- profils candidat et recruteur ;
- préférences et complétion ;
- offres et filtres ;
- matching ;
- profil, photo et CV candidat ;
- logo recruteur ;
- conversations HTTP ;
- messages lus et non lus ;
- notifications ;
- authentification et diffusion Socket.IO ;
- présence des routes dans OpenAPI.

Résultat de la dernière exécution :

13 tests réussis
0 échec

Les tests utilisent deux bases MongoDB locales dédiées :

```
job-swipe-test-auth  
job-swipe-test-workflow
```

15. Historique des évolutions

Les principales évolutions ont été poussées sur **develop** :

- **09016bd** : authentification, profils et offres ;
- **4fac862** : workflow de matching ;
- **f58b6f3** : conversations et messagerie ;
- **3c72658** : gestion des CV ;
- **499e45f** : Socket.IO ;
- **6e48acb** : tests d'intégration ;
- **16612bb** : pipeline CI ;
- **88c66f6** : documentation OpenAPI ;
- **ca300a7** : messages lus et non lus ;
- **3282d97** : notifications ;
- **04f68fa** : complétion des profils ;
- **28b6a6f** : médias et score de matching.

16. Prochaines étapes

Les fonctionnalités encore à traiter sont :

- sécurisation globale du backend ;
- score enrichi et pondérations configurables ;
- distinction **like** et **super like** ;
- statut candidat mis de côté ;
- présence en ligne ;
- signalement de conversation ;
- export RGPD ;
- suppression RGPD complète ;
- tests de sécurité dans la CI.

Conclusion

Le backend couvre maintenant le parcours principal de JobSwipe : un candidat construit son profil, consulte une offre, manifeste son intérêt, puis un recruteur examine les informations utiles avant d'accepter ou refuser. L'acceptation crée un match réciproque, une conversation et les notifications associées.

Le projet dispose également d'une documentation OpenAPI, d'une pipeline CI et d'une couverture de tests automatisés permettant de faire évoluer le backend de manière contrôlée.